

Grafuri. Componente tare conexe

Un graf orientat este tare conex dacă în orice nod al său se poate ajunge din toate celelalte vârfuri. Fie un graf orientat, $G(V, E)$, unde V este mulțimea vârfurilor, iar E mulțimea muchiilor. Dacă G nu este tare conex, el poate fi partiționat în componente tare conexe. Acestea sunt subgrafuri $G_i(V_i, E_i)$ în care există o cale în ambele direcții între orice pereche de vârfuri.

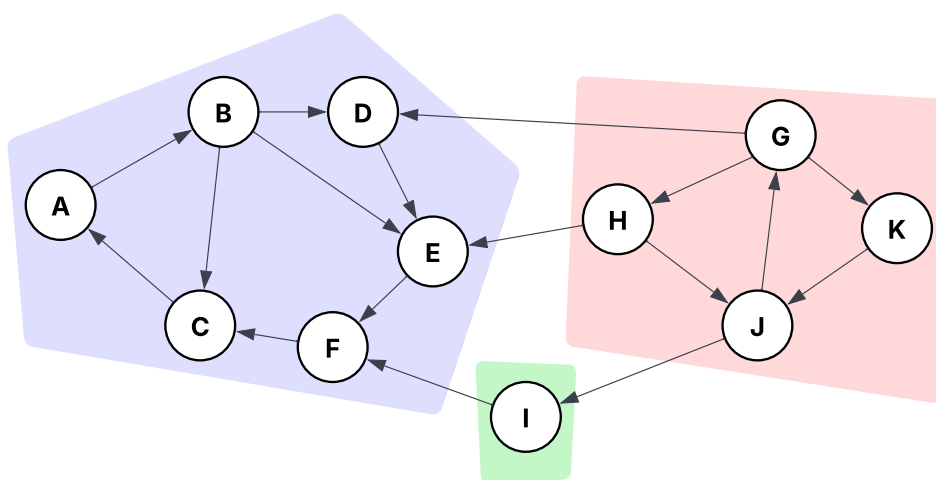


Figura 1: Componente tare conexe

Graful din Figura 1 are trei componente tare conexe. În interiorul componentei albastre putem găsi mai multe subgrafuri tare conexe, de exemplu (A, B, C) , (B, D, E, F, C) etc, însă doar subgraful maximal se consideră a fi o componentă tare conexă.

Observați că niciun nod din componenta roșie nu este accesibil nici din componenta albastră, nici din cea verde. Acest tip de componentă conexă se numește **închisă**.

Observați și că nodurile din componenta albastră sunt accesibile atât din componenta roșie, cât și din cea verde. Componenta verde este, de asemenea, accesibilă din componenta roșie. Astfel de componente conexe, ce sunt accesibile din altele, se numesc componente **deschise**.

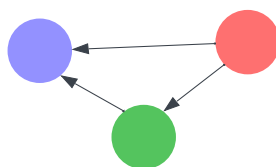


Figura 2: Graful componentelor tare conexe

Dacă toate nodurile dintr-o componentă conexă sunt reduse la un singur vârf, graful rezultat va fi aciclic. Figura 2 corespunde grafului obținut astfel din cel din Figura 1.

1 Parcurgerea în adâncime a grafurilor

Figura 3 prezintă parcurgerea în adâncime (*depth-first search*, DFS) a componentei albastre din graful de mai sus, pornind din vârful B. Numerele asociate nodurilor reprezintă ordinea în care acestea sunt procesate.

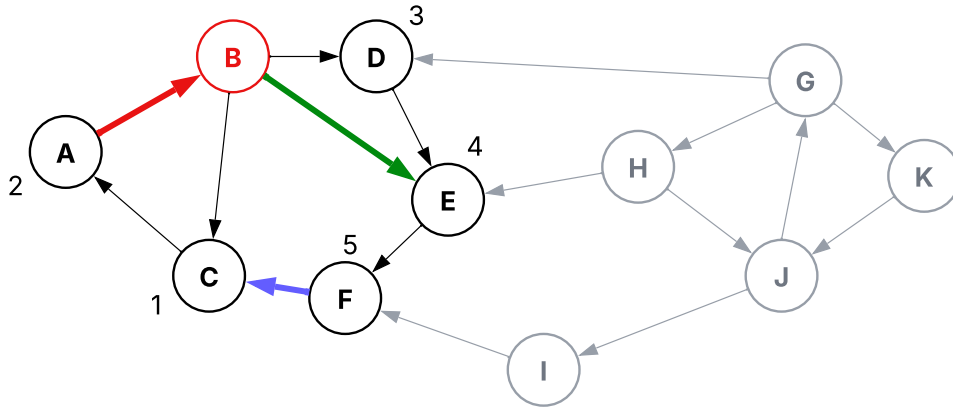


Figura 3: Parcurgerea în adâncime. Muchiile negre sunt *tree edges*, muchia albastră este *cross edge*, muchia verde este *forward edge*, iar muchia roșie este *back edge*.

Așadar, primul vecin vizitat al lui B, din cele trei noduri accesibile din acesta (C,D,E), este vârful C. Parcurgerea urmează cu nodul A, al cărui singur vecin, nodul B, nu va mai fi parcurs, deoarece a fost vizitat deja. Se revine în nodul C, apoi iar în B. Urmează apoi vizitarea vârfurilor E, apoi F. Singurul vecin al lui F este C, care a fost deja vizitat. Se revine în E, apoi în D și din nou în B. Vârful B mai are o legătură către vârful E, dar și acestea a fost deja vizitat, astfel că parcurgerea DFS pentru această componentă conexă se oprește: au fost vizitate toate vârfurile.

- Muchiile negre din Figura 3 reprezintă arcele parcurse prin DFS. Observați că ele alcătuiesc un arbore cu rădăcina în B; se mai numesc și **tree edges**.
- Observați că muchia $A \rightarrow B$ reprezintă o întoarcere de la un nod urmaș (A) la un nod strămoș. În cazul acesta strămoșul este chiar rădăcina arborelui, B. Acest tip de muchii din graf, neexplorate în parcurgerea DFS, se numesc **back edges**.
- Muchia dintre F și C nu a fost parcursă în DFS deoarece nodul C era deja vizitat (vedeți timpii în care au fost parcurse vârfurile C și F). Observați că aceasta muchie a grafului reprezintă o legătură între două ramuri diferite ale arborelui DFS: ramura $B \rightarrow C \rightarrow A$ și ramura $B \rightarrow D \rightarrow E \rightarrow F$. Aceste tipuri de muchii se numesc **cross edges**.
- Muchia dintre B și D nu a fost, de asemenea, parcursă în DFS, deoarece nodul E fusese deja vizitat prin D. Observați că ea este, însă, similară muchiilor din arbore (*tree edges*), în sensul că face legătura între un strămoș și un urmaș. Aceste muchii se numesc **forward edge**.

Parcurgerea în adâncime poate fi implementată cu ajutorul unei stive și al unui vector în care se reține dacă nodurile au fost sau nu vizitate. Se introduce un nod în stivă, apoi se repetă următoarele operații: se extrage un nod din stivă și, dacă nu a fost deja vizitat, se marchează ca vizitat; apoi se adaugă în stivă toți vecinii nodului. Parcurgerea se reia de la început pentru toate vârfurile nevizitate ale grafului.

2 Identificarea componentelor conexe

Observați care va fi rezultatul parcurgerii în adâncime a grafului 1 dacă aceasta începe din vârful G. Spre deosebire de cazul prezentat în secțiunea anterioară, când singurele noduri parcurse din graf erau cele din componenta tare conexă albastră, acum parcurgerea va include și nodurile celorlalte componente conexe.

Prin urmare, o simplă parcurgere în adâncime nu este suficientă pentru identificarea componentelor tare conexe.

Un exemplu de parcurgere a grafului începând cu nodul G este prezentat în Figura 4.

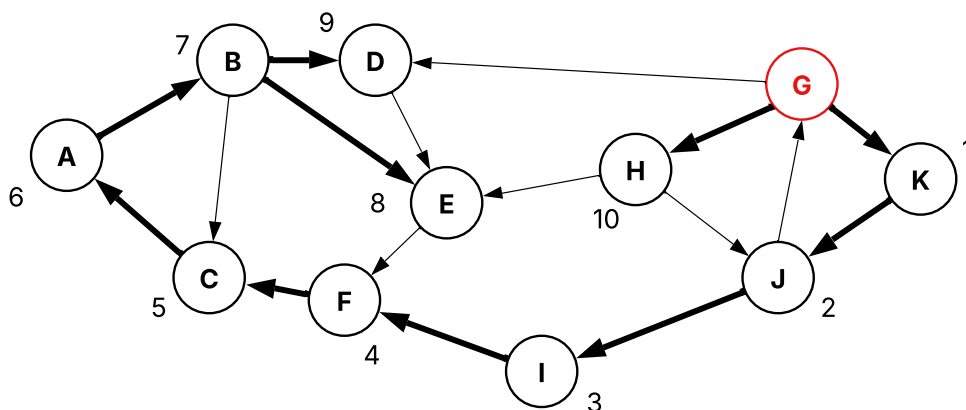


Figura 4: Parcurgerea grafului începând din nodul G. Muchiile negre subțiri nu sunt parcurse, pentru că duc la noduri deja vizitate.

O soluție pentru detectarea componentelor tare conexe presupune identificarea, pe măsură ce graful este parcurs, nodurilor rădăcină pentru propria componentă.

În ordinea de parcurgere DFS din Figura 4, nodul G reprezintă rădăcina componentei conexe roșii, nodul I rădăcina componentei verzi, iar nodul F rădăcina componentei tare conexe albastre.

2.1 Algoritmul lui Gabow

Din G , nodul K este primul vizitat, urmat de nodul J . Considerăm că primul vecin al lui J este nodul G , adică strămoșul lui J . Prezența acestei *back edge* înseamnă că nici K , nici J nu pot fi rădăcini ale unei componente conexe: rădăcina este întotdeauna primul nod parcurs dintr-un arbore. Iar K și J nu pot fi rădăcini, de vreme ce prin ele se poate ajunge la un nod vizitat înaintea lor.

Un alt fel de a vedea lucrurile este că bucla $G \rightarrow K \rightarrow J \rightarrow G$ se poate contracta la un singur nod, G' . Prin contractarea tuturor buclelor întâlnite în graf, acesta este redus la un graf aciclic, spre exemplu la forma din Figura 2.

Gabow [1] propune o implementare a ideii de mai sus care menține o listă a nodurilor-candidat pentru a fi rădăcină pentru componentele conexe. Concret, soluția utilizează:

- două stive: una pentru parcurgerea DFS propriu-zisă (*depth*) și una pentru posibilele noduri candidați (*roots*).
- doi vectori: unul pentru reținerea nodurilor vizitate (ca în DFS) și unul pentru reținerea momentelor de timp la care fiecare nod a fost vizitat, ce va permite identificarea *back edge*-urilor.

Pașii algoritmului sunt următorii:

1. Pe măsură ce nodurile sunt vizitate, acestea sunt introduse în ambele stive, iar vectorii sunt actualizați corespunzător.
2. La întâlnirea unei *back edge*, nodurile conținute în buclă, exceptându-l pe primul vizitat, sunt excluse din stiva de candidați, *roots*. Pasul echivalează cu reducerea nodurilor din buclă la unul singur, care este păstrat în stivă.
3. Identificarea rădăcinilor componentelor conexe se face la **revenirea** din parcurgere. Când se avansează în DFS, prezența buclelor semnalează faptul că ne aflăm în interiorul unei componente conexe. La pasul anterior, buclele sunt reduse la un singur nod, una câte una. Când s-au epuizat toate buclele, nodul rămas este rădăcina componentei conexe.
Dacă **revenind** din parcurgere (/recursivitate) se ajunge la nodul care este în vârful stivei *roots*, înseamnă că toate nodurile accesibile din acesta au fost vizitate. Vârful este, deci, rădăcina unei componente conexe.
4. Operațiile de mai sus se repetă începând cu toate nodurile din graf, la fel ca la parcurgerea DFS.

Anexa A detaliază pașii algoritmului pe graful din Figura 4.

3 Probleme

1. (8p) Scrieți (pe hârtie) pașii aplicării algoritmului Gabow pe graful din Figura 5, începând parcurgerea din vârful G.
2. (2p) Programul [gabow.c](#) conține implementarea algoritmului, cu excepția câtorva pași. Completați codul, rulați-l și verificați că obțineți același rezultat ca la punctul precedent.

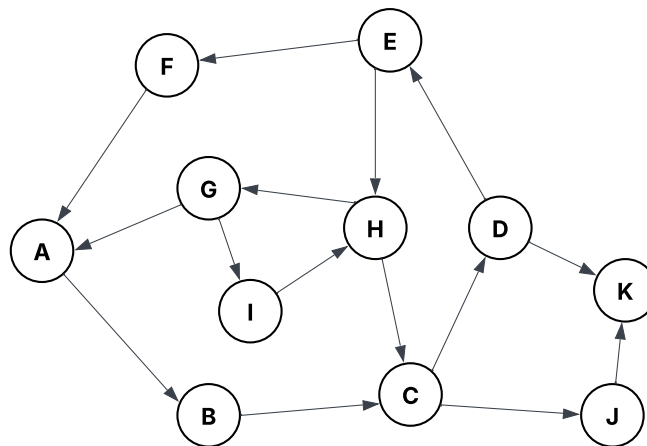


Figura 5: Graf cu componente tare conexe

A Gabow - exemplu

După vizitarea nodurilor G, K, J din Figura 4, conținutul structurilor va fi cel din Tabelul 1.

Tabelul 1: Gabow. Conținutul structurilor după introducerea nodurilor G, K, J

	A	B	C	D	E	F	G	H	I	J	K
vizitat	0	0	0	0	0	0	1	0	0	1	1
timp	-1	-1	-1	-1	-1	-1	0	-1	-1	2	1
stiva <i>depth</i>	G	K	J								
stiva <i>roots</i>	G	K	J								

Primul vecin al lui J este G. Însă G a fost deja vizitat, ceea ce înseamnă că a fost detectată o buclă. Toate nodurile care au fost introduse după G nu pot fi rădăcini ale componentei conexe, așa că se extrag din stiva *roots*. Concret, se extrag din stiva *roots* toate nodurile care au timpi mai mari decât G (care au fost vizitate după G).

Se continuă parcurgerea cu al doilea vecin al nodului J, anume I. De aici se ajunge până la nodul E. Conținutul structurilor va fi cel din Tabelul 2.

Tabelul 2: Gabow. Conținutul structurilor după introducerea nodurilor I-E

	A	B	C	D	E	F	G	H	I	J	K
vizitat	1	1	1	0	1	1	1	0	1	1	1
timp	6	7	5	-1	8	4	0	-1	3	2	1
stiva <i>depth</i>	G	K	J	I	F	C	A	B	E		
stiva <i>roots</i>	G	K	J	I	F	C	A	B	E		

Vecinul lui E este F, care a fost de asemenea vizitat. Din nou, vor fi extrase din stiva *roots* toate nodurile care au timpi mai mari decât ai lui F, adică cele vizitate după momentul 4. Ele nu pot fi rădăcini ale componentei conexe.

Parcurgerea continuă prin revenirea în B și alegerea unui alt vecin al acestuia. Presupunem că al doilea vecin e nodul C. Însă C a fost vizitat, la momentul 5. Acum ar trebui extrase din stiva *roots* toate nodurile la care s-a ajuns după C, însă nu mai există în *roots* astfel de noduri.

Se revine din nou în B și se continuă parcurgerea în D. Singurul vecin al lui D este E, care a fost vizitat, deci se extrag din stiva *roots* toate nodurile la care s-a ajuns după E, adică nodul D. În acest moment, conținutul structurilor este cel din Tabelul 3.

Tabelul 3: Gabow. Conținutul structurilor după vizitarea nodului D

	A	B	C	D	E	F	G	H	I	J	K
vizitat	1	1	1	1	1	1	1	0	1	1	1
timp	6	7	5	9	8	4	0	-1	3	2	1
stiva <i>depth</i>	G	K	J	I	F	C	A	B	E	D	
stiva <i>roots</i>	G	K	J	I	F	C	A	B	E	D	

Se revine din parcurgere la nodul B, apoi în A, C, F. Observați că F este vârful stivei *roots*. Acest lucru indică faptul că F este rădăcina componentei conexe care a fost doar ce parcursă: este primul strămoș al nodurilor din componenta conexă.

Acum pot fi extrase din stiva *depth* toate nodurile din componenta conexă cu rădăcina F, inclusiv acesta, ca în Tabelul 4.

Tabelul 4: Gabow. Conținutul structurilor după aflarea primei componente conexe

	A	B	C	D	E	F	G	H	I	J	K
vizitat	1	1	1	1	1	1	1	1	1	1	1
timp	6	7	5	9	8	4	0	10	3	2	1
stiva <i>depth</i>	G	K	J	I	F	C	A	B	E	D	
stiva <i>roots</i>	G	K	J	I	F	C	A	B	E	D	

Se revine din nou din parcurgere, în nodul I. Acesta este iarăși vârful stivei *roots*, deci este rădăcina unei componente conexe și va fi extras din stiva *depth*.

Revenirea continuă cu nodurile J, K, G și parcurgerea se continuă cu nodul H, care este marcat ca vizitat. Primul vecin al lui H este J, care a fost vizitat, deci a fost detectată o nouă buclă. Vor fi extrase din stiva *roots* nodurile care au fost vizitate după J, adică H.

La revenirea din H se ajunge în nodul G, care este acum vârful stivei *roots*. Prin urmare, s-a detectat o nouă rădăcină a unei componente tare conexe și se pot extrage din stiva *depth* toate nodurile rămase: G, K, J, H.

Tabelul 5: Gabow. Conținutul structurilor la revenirea din nodul H

	A	B	C	D	E	F	G	H	I	J	K
vizitat	1	1	1	1	1	1	1	1	1	1	1
timp	6	7	5	9	8	4	0	10	3	2	1
stiva <i>depth</i>	G	K	J	I	F	C	A	B	E	D	H
stiva <i>roots</i>	G	K	J	I	F	C	A	B	E	D	H

Bibliografie

- [1] Hardold Gabow. Path-based depth-first search for strong and biconnected components. *Information Processing Letters*, 2000.